

Кратко в предыдущих сериях

Советуем перед семинаром освежить в памяти из курса статистики:

- Описательные статистики: среднее, частота абсолютная и относительная, выборочные медиана и квантили, их поиск с помощью пакета `numpy`
- ЦПТ
- Точечные и интервальные оценки
- Оценки параметров среднего и пропорции
- Визуализации с помощью `matplotlib` и `seaborn`
- Работа с датасетами в `pandas`

Тема 3. Хорошая метрика. Кто она?

0. Цель и задачи

Цель — разобраться, как выбрать метрику под конкретную задачу

Задачи:

- повторить, какие есть основные свойства метрик
- посмотреть на практике на некоторые свойства метрик
- узнать, бывают ли идеальные метрики
- потренироваться с выбором метрики под конкретный случай

1. Предисловие

1.1. Вспомним лекцию

Какие свойства метрик вы изучали на лекции?

- **Чувствительность** — метрика быстро и заметно реагирует на изменения в продукте или поведении пользователей
- **Достоверность** — Можем ли мы доверять данным, на которых собирается метрика? Есть ли смещение? Нет ли ошибки выжившего?
- **Эффективность расчета** — Выгодно ли рассчитывать метрику? Принесет ли она больше пользы, чем потратим на ее подсчет?
- **Отлаживаемость** — Можем ли мы понять причины изменения метрики?
- **Интерпретируемость** — Понятно, изменение метрики это хорошо или плохо? Понятно, какие решения принимают, когда увидят изменение метрики? Можем объяснить метрику бизнесу?

1.2. 🍌 Практика: по каждому пункту приведем примеры

По каждому из пунктов давайте приведем примеры и антипримеры таких метрик

- Чувствительность:
 - пример: метрики посещаемости (DAU, MAU, etc) — зачастую даже за небольшой промежуток времени видна серьезная разница
 - антипример: средний чек — очень подвержен выбросам, сложно повышать путем каких-то изменений
- Достоверность:
 - пример: метрики скорости работы сайта — собираются автоматически, полные, редко происходят сбои в сборе
 - антипример: NPS / CSAT — зачастую добровольный, поэтому малое покрытие и нерепрезентативная выборка (отвечают в основном либо "промоутеры", либо "критики")

- Эффективность расчета:
 - пример: метрики скорости работы сайта — собираются автоматически, нужно лишь однократно настроить логику сборки, а пользы могут принести очень много
 - антипример: большие опросы удовлетворенности сотрудников каждую неделю (10-15 минут) — если такой опрос выкатить в Т, то компания будет терять десятки миллионов рублей просто на времени прохождения опроса. А получение расширенной еженедельной обратной связи имеет сомнительную ценность
- Отлаживаемость:
 - пример: метрики удержания — хорошие изменения → сохраненный клиент
 - антипример: метрики посещаемости — на них влияет все: и сезонность, и случайный рост продукта, часто непонятно, отчего произошли изменения
- Интерпретируемость:
 - пример: конверсия в покупку — много покупают = хорошо
 - антипример: кол-во шагов для покупки — не всегда понятно, много — это хорошо (заинтересованность, подробное изучение) или плохо (сложный UX)

Вот таблица по основным метрикам и их характеристикам:

Метрика	Чувствительность	Достоверность	Отлаживаемость	Интерпретируемость
Метрики посещаемости	Зачастую высокая — часто довольно легко повлиять даже за небольшой промежуток времени	Зачастую высокая, но может быть фрод / дубликаты	Низкая — рост может быть связан с чем угодно: маркетинг, сезон, случайность	Высокая — все понимают, что рост это хорошо
Метрики удержания	Средняя — чувствительна к UX-изменениям, но сложно с замером роста на маленьких данных	Зачастую высокая, но могут быть пропуски при неполной сборке событий	Высокая — можно строить анализ когорт, кривые ретеншна и понимать, за счет чего происходят изменения	Средняя — не всегда очевидно, почему удержание падает и как это связано с продуктом
Конверсия в регистрацию	Средняя — чувствительна к UX-изменениям, но сложно с замером роста на маленьких данных	Зачастую высокая, но может быть фрод / дубликаты	Высокая — можно смотреть по шагам воронки	Высокая — легко объяснить влияние изменений UI/UX
Конверсия в оплату	Средняя — часто довольно маленькие значения, поэтому сложно заметить изменения	Средняя — страдает от узкой выборки (только активные клиенты)	Средняя — не всегда ясно, что влияет: высокая/низкая цена, user-friendly/not friendly интерфейс	Высокая — оплатили = хорошо
Средний чек (ARPU)	Низкая — чувствительна к выбросам и малому числу платящих	Средняя — могут исказить скидки, возвраты, ошибки, плюс смотрит только на платящих (ошибка выжившего)	Низкая — тяжело объяснить рост: больше покупок или товар подорожал?	Средняя — мало показывает без кол-ва покупок, очень важно смотреть динамику
Кол-во багов/ жалоб	Высокая — исправили баг, и жалуются обычно меньше 😊	Низкая — зависит от того, кто жалуется, и как фиксируются баги	Высокая — можно группировать по типам ошибок, продуктам	Средняя — жалоб много, это плохо, но может быть, мало жалоб = равнодушие к продукту
NPS / CSAT	Низкая — инертна, чувствительна к формулировке вопросов, может быть малое число респондентов	Низкая — зачастую добровольный, поэтому малое покрытие и нерепрезентативная выборка (ошибка выжившего)	Низкая — не всегда понятно, что повлияло: уменьшение цены? улучшение продукта/ сервиса?	Средняя — бизнес любит, но сложно связать с бизнес-ценностью
Кол-во просмотренных экранов / шагов	Средняя — зависит от продукта	Зачастую высокая, но могут быть пропуски при неполной сборке событий	Средняя — плохо декомпозируется, так как уже низкоуровневая	Низкая — не всегда понятно, много — это хорошо (заинтересованность, подробное изучение) или плохо (сложный UX)
Метрики загрузки / скорости	Зачастую высокая — даже небольшие задержки заметны, но смотрим не среднее, а перцентиль (имеет довольно тяжелые хвосты)	Высокая — собираются автоматически, полные, редко происходят сбои в сборе	Высокая — легко найти виновника: сервер, сеть, клиент	Средняя — бизнесу понятно: быстрее = лучше, но не всегда быстрая загрузка = профит

2. Закрепим на примерах

Давайте потренируемся работать с некоторыми свойствами метрик на практике

Ввод [74]:

```
1 import matplotlib.pyplot as plt
2 import matplotlib.dates as mdates
3 import scipy.stats as sps
4 from scipy.stats import zscore
5 import numpy as np
6 import seaborn as sns
7 import pandas as pd
8 from tqdm.notebook import tqdm
9 from statsmodels.tsa.seasonal import STL, seasonal_decompose
10 import os
11 from pylab import rcParams
12 from sklearn.linear_model import LinearRegression
13 from datetime import datetime, timedelta
14 from joblib import Parallel, delayed
15
16 sns.set_theme(style='whitegrid')
17 sns.set(palette="Set2", font_scale=1)
```

2.1 Чувствительность

Начнем с чувствительности.

👉 Интерактив: как можно оценить чувствительность?

Студенты предлагают, что можно сделать. Самые логичные ответы — посмотреть длину дов. интервалов или, если у студентов высокий уровень, провести аб-тест на истории и оценить мощность (последнее — будем делать позже, а далее в семинаре будет первый решение по первому ответу).

Типовой ответ: — Посмотреть длину дов. интервалов. Если у метрики широкий интервал, значит, на эксперименте, скорее всего, мы не заметим изменения. — Широкий в абсолютных или относительных значениях? — В относительных. Например, у конверсии в просмотр товара конверсия 0.6 ± 0.02 , а у конверсии в покупку конверсия 0.05 ± 0.01 . Очевидно, что конверсию в просмотр увеличить на 2 п.п. легче, чем конверсию в покупку на 1 п.п.

Ввод [2]:

```
1 tqdm.pandas()
```


Ввод [5]:

```

1 # Metric Sensitivity & Reliability Demo – Synthetic Dataset and Quick Analysis
2 # -----
3 # v2 – Adds:
4 #   • Operating-system dimension (iOS 30 % / Android 70 %).
5 #   • Fixed session-date range 01-Jun-2024 → 31-Aug-2024 with weekly seasonality
6 #     (more sessions on Fri/Sat/Sun).
7 #   • iOS sessions have higher AOV; Android sessions suffer a purchase-rate drop
8 #     starting 1-Aug-2024 (simulating a bug or payment issue) but otherwise share
9 #     the same baseline conversion rates.
10 #   • All previous outlier / CSAT bias logic retained.
11 #
12 # Run the script top-to-bottom; the two comparison tables at the bottom illustrate
13 #   1) sensitivity of mean AOV to spend outliers, and
14 #   2) survivorship-biased CSAT averages.
15 # -----
16
17 # RNG for reproducibility
18 rng = np.random.default_rng(42)
19
20 # -----
21 # 1. Synthetic session-level data
22 # -----
23
24 N_USERS = 500_000 + rng.choice(100_000)
25 print(N_USERS)
26 sessions_per_user = rng.poisson(lam=6, size=N_USERS) + 1 # ≥1 session per user
27
28 START_DATE = datetime(2024, 6, 1)
29 END_DATE = datetime(2024, 8, 31)
30 TOTAL_DAYS = (END_DATE - START_DATE).days + 1
31
32 # Weekly seasonality weights (Mon→Sun): fewer sessions Mon–Thu, peak Sat/Sun
33 weekday_weights = np.array([1.0, 0.9, 1.05, 1.1, 1.3, 1.5, 1.5])
34 # Build per-day probabilities over the 3-month span
35 weights = np.zeros(TOTAL_DAYS)
36 for d in tqdm(range(TOTAL_DAYS)):
37     day = START_DATE + timedelta(days=d)
38     weights[d] = weekday_weights[day.weekday()]
39 weights /= weights.sum() # normalise
40
41 records: list[tuple] = []
42 session_id = 0
43
44 for user_id, n_sess in tqdm(enumerate(sessions_per_user, start=1)):
45     # heterogeneous per-user propensities
46     p_view = rng.uniform(0.25, 0.9)
47     p_cart = rng.uniform(0.15, 0.6)
48     p_purchase = rng.uniform(0.05, 0.35)
49
50     for _ in range(n_sess):
51         session_id += 1
52
53         # --- OS split -----
54         os = rng.choice(["iOS", "Android"], p=[0.3, 0.7])
55
56         # --- Date with seasonality -----
57         day_offset = rng.choice(TOTAL_DAYS, p=weights)
58         date = START_DATE + timedelta(days=int(day_offset))
59
60         # --- Funnel simulation -----
61         viewed = rng.random() < p_view
62         added = viewed and (rng.random() < p_cart)
63
64         # baseline purchase prob
65         p_buy = p_purchase if added else 0.0
66         # Android drop after 1-Aug-2024
67         if os == "Android" and date >= datetime(2024, 8, 1):
68             p_buy *= 0.5 # 50 % relative drop
69         purchased = rng.random() < p_buy
70
71         # --- Spend -----

```

```

72     spend = 0.0
73     if purchased:
74         spend = rng.gamma(shape=3, scale=700) # mean≈40
75         # Rare big-spender
76         if rng.random() < 0.010:
77             spend *= 20
78         # iOS users spend more on average
79         if os == "iOS":
80             spend *= 1.7 # +70 % AOV uplift
81
82     base_prob = 0.05 # 5 % chance overall
83     if purchased:
84         base_prob = 0.40
85     elif viewed:
86         base_prob = 0.08
87
88     answered = rng.random() < base_prob
89     mean_score = 4.2 if purchased else (3.5 if viewed else 3.0)
90     csat_score = np.nan if not answered else round(np.clip(rng.normal(loc=mean_score, scale=0.5), 1, 5), 1)
91     records.append(
92         (
93             user_id,
94             session_id,
95             os,
96             date.date(), # keep as date for clarity
97             viewed,
98             added,
99             purchased,
100             round(spend, 2),
101             answered,
102             csat_score
103         )
104     )
105
106     cols = [
107         "user_id",
108         "session_id",
109         "os",
110         "session_date",
111         "viewed",
112         "added",
113         "purchased",
114         "spend",
115         "csat_answered",
116         "csat_score"
117     ]
118
119     df = pd.DataFrame.from_records(records, columns=cols)

```

508925

100%

92/92 [00:00<00:00, 21648.02it/s]

508925/? [01:13<00:00, 6030.91it/s]

Ввод [6]: 1 df

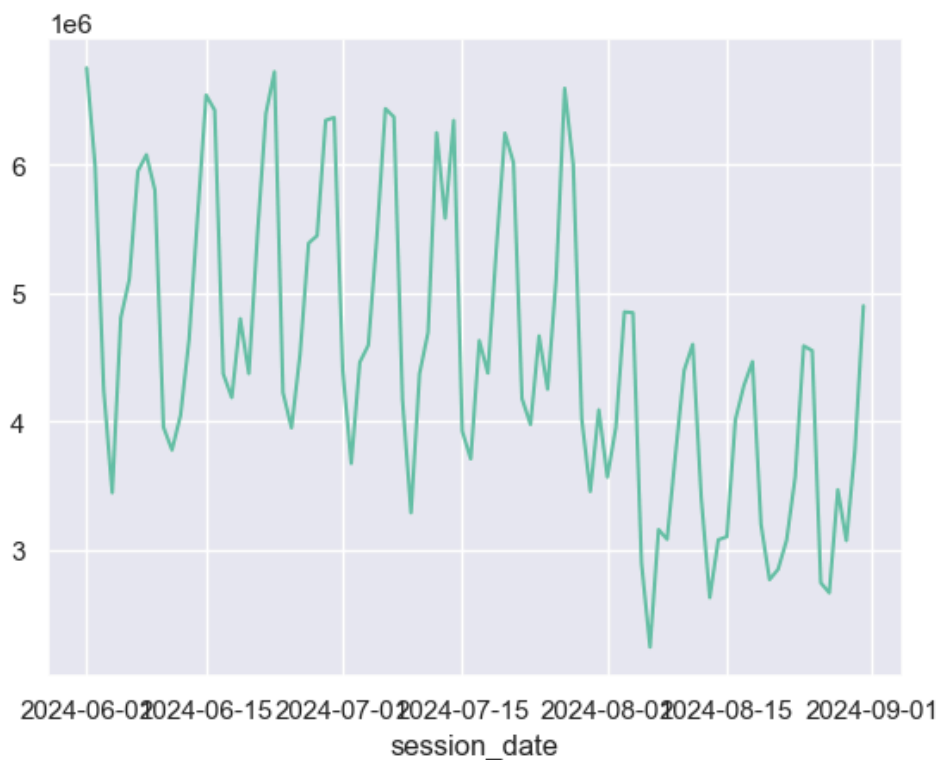
Out[6]:

	user_id	session_id	os	session_date	viewed	added	purchased	spend	csat_answered	csat_sc
0	1	1	Android	2024-07-20	True	False	False	0.00	False	М
1	1	2	iOS	2024-08-30	False	False	False	0.00	False	М
2	1	3	Android	2024-06-23	False	False	False	0.00	False	М
3	1	4	Android	2024-08-28	True	True	False	0.00	False	М
4	1	5	Android	2024-07-04	True	True	True	2049.39	False	М
...	...	...	...	...	...	...	...	...	...	...
3562279	508925	3562280	Android	2024-06-17	False	False	False	0.00	False	М
3562280	508925	3562281	Android	2024-06-04	True	False	False	0.00	False	М
3562281	508925	3562282	iOS	2024-06-16	True	False	False	0.00	False	М
3562282	508925	3562283	Android	2024-07-01	False	False	False	0.00	False	М
3562283	508925	3562284	Android	2024-08-02	False	False	False	0.00	False	М

3562284 rows × 10 columns

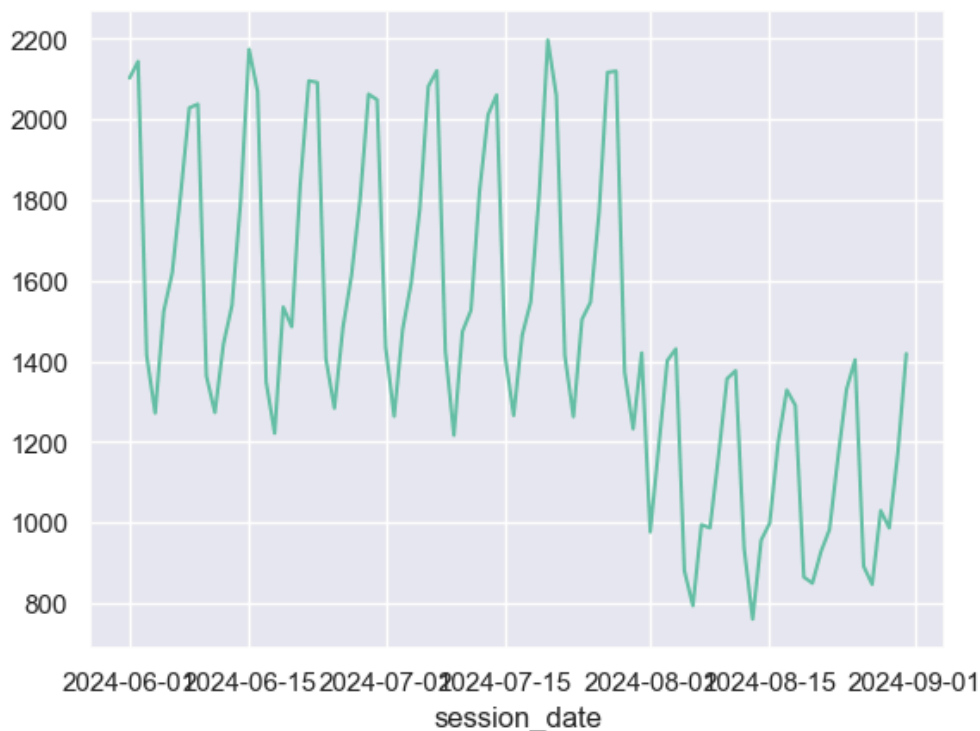
Ввод [9]: 1 df.groupby('session_date')['spend'].sum().plot()

Out[9]: <Axes: xlabel='session_date'>



```
Ввод [11]: 1 df.groupby('session_date')['purchased'].sum().plot()
```

```
Out[11]: <Axes: xlabel='session_date'>
```



```
Ввод [12]: 1 df[df['purchased']]['spend'].mean()
```

```
Out[12]: 3081.947515830133
```

```
Ввод [13]: 1 df[df['purchased']][df['os'] == 'Android']['spend'].mean()
```

```
/var/folders/ym/2l1gn_s5ss0_f4qd4t25z500000gq/T/ipykernel_7849/3780170145.py:1: U
serWarning: Boolean Series key will be reindexed to match DataFrame index.
df[df['purchased']][df['os'] == 'Android']['spend'].mean()
```

```
Out[13]: 2488.610515043459
```

```
Ввод [14]: 1 df[df['purchased']][df['os'] == 'iOS']['spend'].mean()
```

```
/var/folders/ym/2l1gn_s5ss0_f4qd4t25z500000gq/T/ipykernel_7849/2167380447.py:1: U
serWarning: Boolean Series key will be reindexed to match DataFrame index.
df[df['purchased']][df['os'] == 'iOS']['spend'].mean()
```

```
Out[14]: 4241.512455192839
```

```
Ввод [15]: 1 df.to_csv('market.csv', index=False)
```

Рассмотрим датасет с сессиями в интернет-магазине за некоторое время.

Приведем небольшое описание колонок датасета:

Описание колонок в сгенерированном датасете

Название	Описание
user_id	ID пользователя (уникален на уровне пользователя)
session_id	ID сессии (уникален глобально)
viewed	Флаг (bool), был ли просмотр хотя бы одного товара в сессии
added	Флаг (bool), был ли добавлен хотя бы один товар в корзину
purchased	Флаг (bool), была ли совершена покупка в этой сессии
spend	Сумма покупки в рублях (0, если не было покупки)

Название	Описание
csat_answered	Флаг (bool), был ли заполнен CSAT-опрос по итогам сессии

Посмотрим на данные

Ввод [16]:

```
1 df = pd.read_csv('market.csv')
2 df
```

Out[16]:

	user_id	session_id	os	session_date	viewed	added	purchased	spend	csat_answered	csat_sc
0	1	1	Android	2024-07-20	True	False	False	0.00	False	NaN
1	1	2	iOS	2024-08-30	False	False	False	0.00	False	NaN
2	1	3	Android	2024-06-23	False	False	False	0.00	False	NaN
3	1	4	Android	2024-08-28	True	True	False	0.00	False	NaN
4	1	5	Android	2024-07-04	True	True	True	2049.39	False	NaN
...	...	...	...	...	...	...	...	...	...	...
3562279	508925	3562280	Android	2024-06-17	False	False	False	0.00	False	NaN
3562280	508925	3562281	Android	2024-06-04	True	False	False	0.00	False	NaN
3562281	508925	3562282	iOS	2024-06-16	True	False	False	0.00	False	NaN
3562282	508925	3562283	Android	2024-07-01	False	False	False	0.00	False	NaN
3562283	508925	3562284	Android	2024-08-02	False	False	False	0.00	False	NaN

3562284 rows × 10 columns

👉 Интерактив. Какие метрики будем сравнивать?

Студенты предлагают метрики, которые можно сравнить. Лучше всего сравнить конверсию в покупку и средний чек (среди купивших.)

```
1 Возьмем функции из предыдущего семинара по бутстрепу
2
3 > Напомним, что мы не можем рассматривать сессии как независимые события, так
  как у одного пользователя могут быть несколько, вообще говоря, зависимых между
  собой сессий. Именно поэтому важно в бутстрепе учесть сэмплирование по
  пользователям.
```

```

Ввод [27]: 1 def run_1_iteration(data, target_col, boot_fn, stat_fn):
2           data_boot = boot_fn(data, target_col)
3           return stat_fn(data_boot)
4
5
6 def count_bootstrap_statistics(data, target_col, boot_fn, stat_fn, n_boot=10_000):
7     """
8     Подсчет бутстрепных статистик методом подстановки
9
10    Параметры:
11        data -- датафрейм с выборкой
12        target_col -- колонка с интересующей нас метрикой
13        boot_fn -- функция, реализующая бутстреп
14        stat_fn -- функция, считающая статистику
15        n_boot -- кол-во запусков бутстрепа
16
17    Возвращает:
18        boot_stat -- массив бутстрепных статистик
19    """
20
21    boot_stat = Parallel(n_jobs=n_jobs)(delayed(run_1_iteration)(
22        data, target_col, boot_fn, stat_fn)
23        for _ in tqdm(range(n_boot), leave=False))
24
25    return boot_stat

```

```

Ввод [28]: 1 def bootstrap_users(data, target_col, user_col='customer_id'):
2     """
3     Делает одну бутстрепную выборку пользователей и возвращает все их заказы.
4
5     Параметры:
6         data -- датафрейм с выборкой
7         target_col -- колонка с интересующей нас метрикой
8         user_col -- колонка с идентификатором пользователя.
9
10    Возвращает:
11        boot_data -- новый датафрейм с бутстреп-выборкой заказов.
12    """
13
14    # Получаем список уникальных пользователей
15    unique_users = data[user_col].unique()
16
17    # Бутстрепим пользователей с возвращением
18    sampled_users = np.random.choice(unique_users, size=len(unique_users), replace=True)
19
20    # Собираем все заказы выбранных пользователей
21    boot_data = data[data[user_col].isin(sampled_users)].copy()
22
23    # Добавим, сколько раз каждый пользователь попал в выборку (важно для повторения)
24    user_counts = pd.Series(sampled_users).value_counts()
25
26    # Повторим строки в соответствии с количеством выборов пользователя
27    boot_data = boot_data.merge(user_counts.rename("replications"), left_on=user_col, right_index=True, how="left")
28    boot_data = boot_data.loc[boot_data.index.repeat(boot_data['replications'])]
29
30    return boot_data

```

Напишем функцию для визуализации доверительных интервалов

Ввод [29]:

```

1 def count_conf_ints(df, target_col, user_col, name_metric, alpha=0.95, n_boot=1000)
2     """
3     Строит и сравнивает классический и бутстрэп-доверительные интервалы (ДИ)
4     для среднего значения метрики. Визуализирует распределение бутстрэпных сред
5     и длину интервалов.
6
7     Параметры
8     -----
9     df : pandas.DataFrame
10         Датасет
11
12     target_col : str
13         Название колонки с метрикой метрики
14
15     alpha : float, optional (по умолчанию = 0.95)
16         Уровень доверия для построения интервалов (например, 0.95 – это 95% ДИ)
17
18     n_boot : int, optional (по умолчанию = 10_000)
19         Количество итераций бутстрэпа
20
21     precision : int, optional (по умолчанию = 0)
22         Количество знаков после запятой при округлении значений для вывода
23
24     Вывод
25     -----
26     На экран печатается:
27     – Среднее значение метрики
28     – Бутстрэп-доверительный интервал
29     – Относительная длина дов. интервала (в долях от среднего значения)
30
31     Также строится гистограмма распределения бутстрэпных средних значений
32     с визуализацией границ обоих интервалов.
33
34     Примечания
35     -----
36     – Классический ДИ основан на t-распределении с оценкой стандартной ошибки.
37     – Бутстрэп-интервал строится по квантилям распределения бутстрэпных средних
38     – Используется визуализация на matplotlib для наглядного сравнения.
39
40     Возвращает
41     -----
42     None
43     """
44     # --- Среднее ---
45     mean = df[target_col].mean()
46
47     # --- Бутстрэп ДИ ---
48     bootstrap_means = count_bootstrap_statistics(
49         df, target_col=target_col, stat_fn=np.mean, n_boot=n_boot,
50         boot_fn=lambda df, target_col: bootstrap_users(df, target_col,
51                                                         user_col=user_col))
52     ci = (np.quantile(bootstrap_means, (1 - alpha) / 2),
53           np.quantile(bootstrap_means, (1 + alpha) / 2))
54
55     # --- Вывод ---
56     print(f"Среднее по метрике {name_metric}: {mean:.4f}")
57     print(f"Бутстрэп 95% ДИ:      ({ci[0]:.4f}, {ci[1]:.4f})")
58     print(f"Относительная длина бутстрэпного ДИ: {(ci[1] - ci[0]) / mean:.3f}")
59
60
61     # --- Визуализация ---
62     plt.figure(figsize=(10, 6))
63     plt.hist(bootstrap_means, bins=50, color='skyblue', edgecolor='black', alpha=0.5,
64             density=True)
65     plt.axvline(mean, color='blue', linestyle='--', label=f'Среднее: {mean:.3f}')
66     plt.axvline(ci[0], color='red', linestyle='--',
67                 label=f'Бутстрэп {int(100 * alpha)}% ДИ')
68     plt.axvline(ci[1], color='red', linestyle='--')
69     plt.title(f"Доверительный интервал среднего метрики {name_metric}")
70     plt.xlabel("Среднее значение (из бутстрэп-выборок)")

```

```

72 plt.ylabel("Частота")
73 plt.legend()
74 plt.grid(True)
75 plt.show()
76 return

```

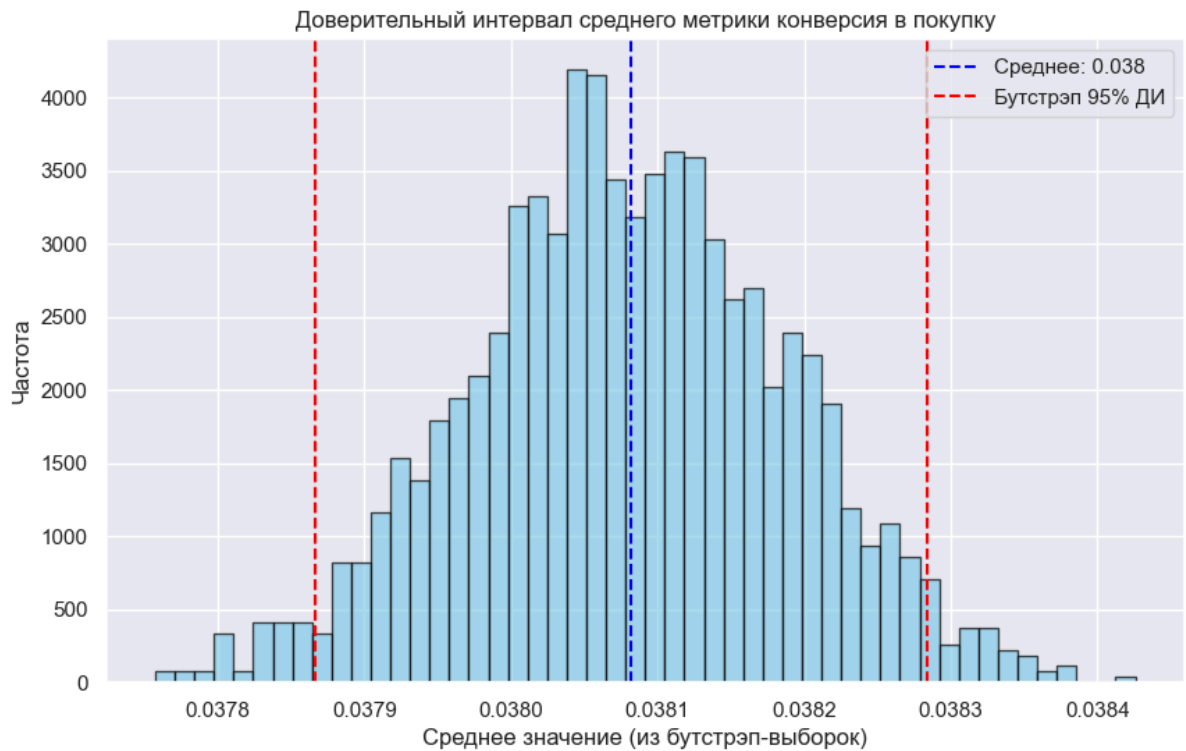
Посчитаем и визуализируем дов. интервалы для конверсии в покупку.

Ввод [33]: 1 count_conf_ints(df, 'purchased', 'user_id', 'конверсия в покупку', n_boot=2_000)

Среднее по метрике конверсия в покупку: 0.0381

Бутстрэп 95% ДИ: (0.0379, 0.0383)

Относительная длина бутстрепного ДИ: 0.011

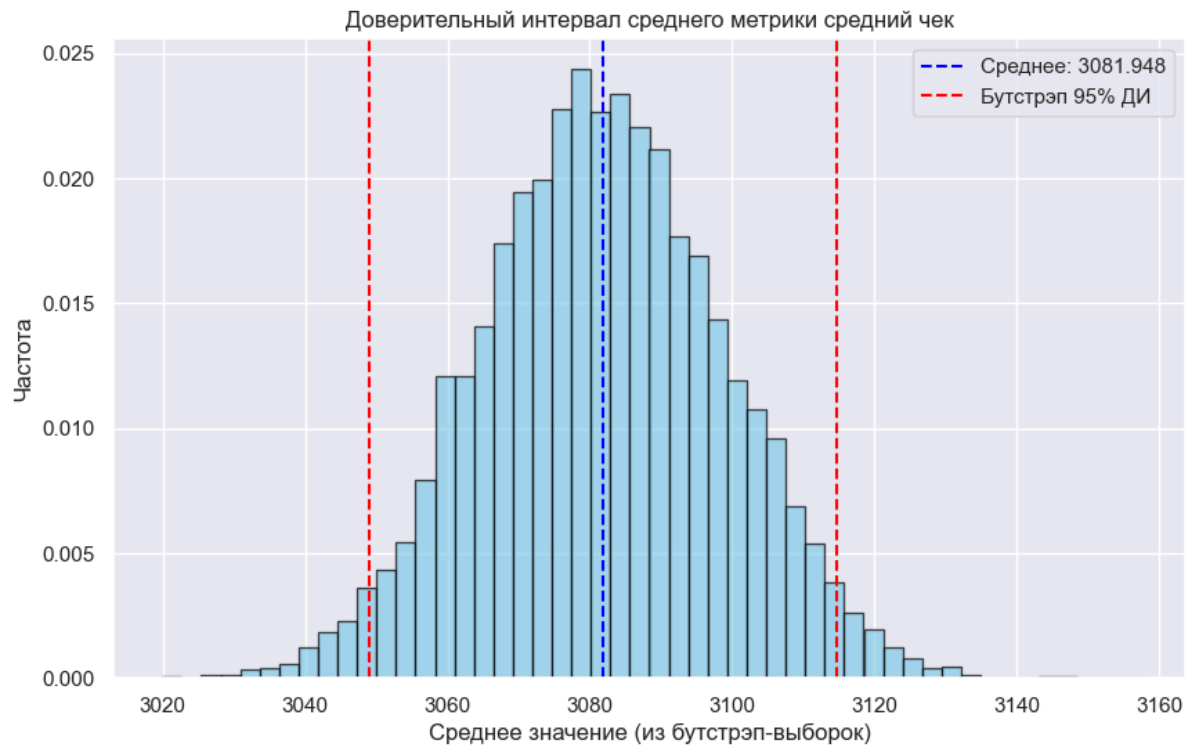


Посчитаем и визуализируем дов. интервалы для среднего чека.

Ввод [34]:

```
1 df_purchased = df[df['purchased']]
2 count_conf_ints(df_purchased, 'spend', 'user_id', 'средний чек', n_boot=10_000)
```

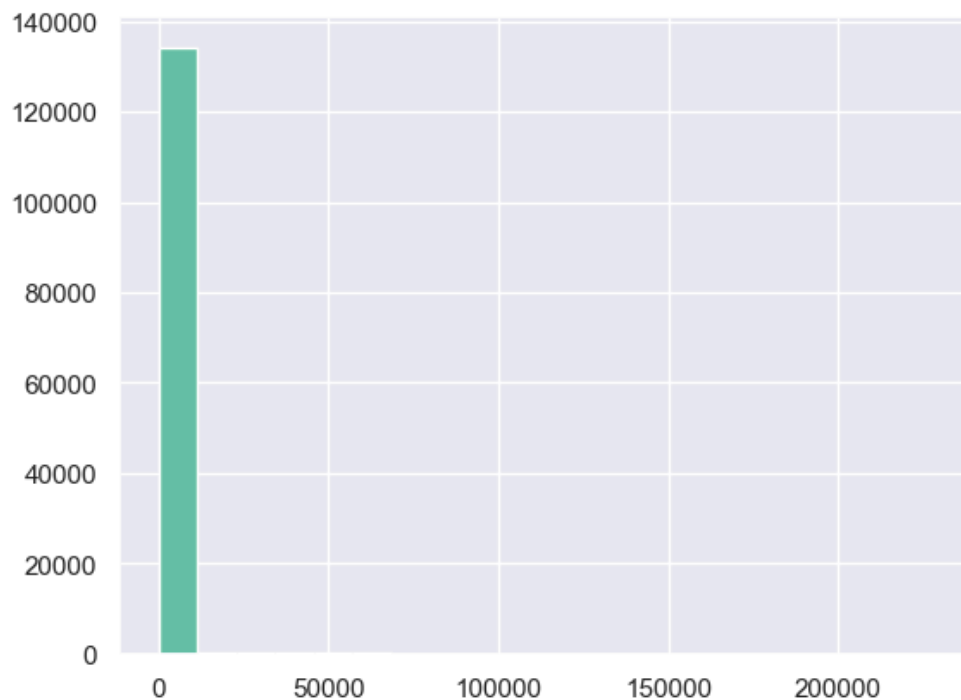
Среднее по метрике средний чек: 3081.9475
 Бутстрэп 95% ДИ: (3048.9640, 3114.7555)
 Относительная длина бутстрепного ДИ: 0.021



Ввод [35]:

```
1 df_purchased['spend'].hist(bins=20)
```

Out[35]: <Axes: >



Ввод [36]:

```
1 df_purchased['spend'].max()
```

Out[36]: 228477.12

```
Ввод [37]: 1 df_purchased['spend'].median(), df_purchased['spend'].mean()
```

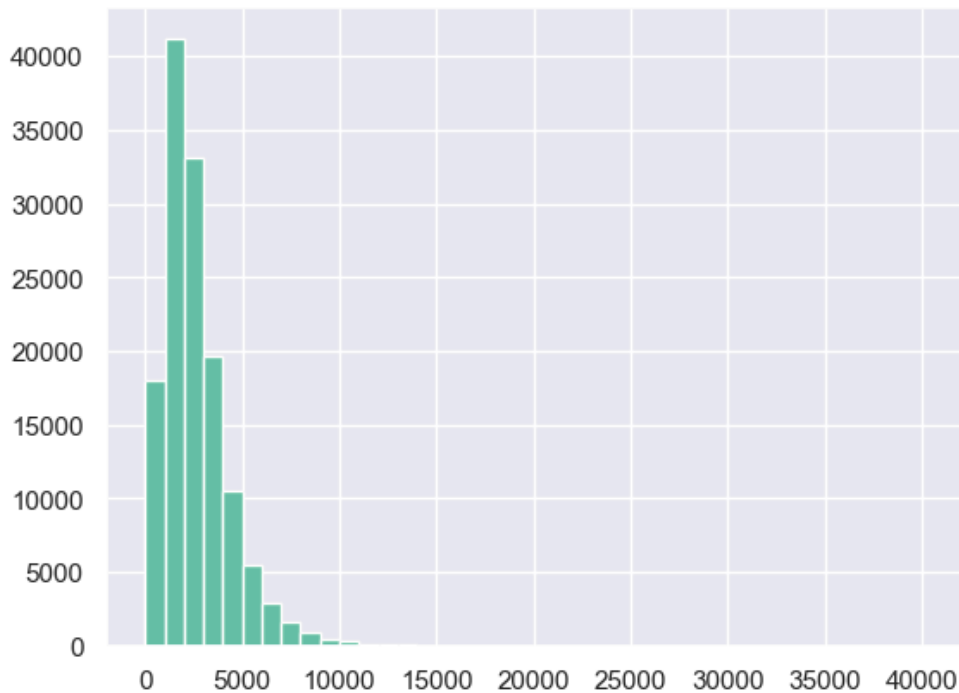
```
Out[37]: (2223.54, 3081.947515830133)
```

```
Ввод [38]: 1 df_purchased['spend'].quantile(0.95)
```

```
Out[38]: 6224.446999999999
```

```
Ввод [39]: 1 df_purchased['spend'].hist(bins=np.linspace(0, 40_000, 41))
```

```
Out[39]: <Axes: >
```



```
Ввод [40]: 1 df['purchased'].mean()
```

```
Out[40]: 0.03808202827174925
```

Вывод: в данном примере более чувствительным оказалась конверсия в покупку: длина ее доверительного интервала 1.1% против 2.1% у среднего чека. Это произошло в основном из-за двух моментов:

- у среднего чека меньше наблюдений: средняя конверсия в покупку 3.8%, что при 3.5 млн сессиях дает нам чуть более 100 тыс. покупок, что значительно меньше наблюдений конверсий в покупку
- у среднего чека есть выбросы: при медиане чуть более 2 тыс. ₽ и среднем чуть больше 3 тыс. ₽ встречается значение более 200 тыс. ₽, при этом 95% чеков меньше 6.5 тыс. ₽

2.2 Достоверность

👉 **Интерактив: что можно проверить на данных из достоверности?**

Из достоверности на данных можно проверить несколько вещей:

- можем ли мы доверять данные — посмотреть Sanity Check из семинара 2
- есть ли смещение/ошибка выжившего? — это мы и будем смотреть на примере

Посмотрим на средний CSAT по всем, по купившим и по просмотревшим

Ввод [41]:

```

1 df_purchased = df[df['purchased']]
2 df_viewed = df[df['viewed']]
3
4 csat = df['csat_score'].mean()
5 csat_purchased = df_purchased['csat_score'].mean()
6 csat_viewed = df_viewed['csat_score'].mean()
7
8 print(f'''средний ксат по всей выборке: {csat:.2f}''')
9 print(f'''средний ксат по купившим: {csat_purchased:.2f}''')
10 print(f'''средний ксат по просмотревшим: {csat_viewed:.2f}''')

```

средний ксат по всей выборке: 3.48
 средний ксат по купившим: 4.13
 средний ксат по просмотревшим: 3.66

Посмотрим также на покрытие CSAT по этим же сегментам

Ввод [42]:

```

1 csat_frac = df['csat_answered'].sum() / len(df)
2 csat_frac_purchased = df_purchased['csat_answered'].sum() / len(df_purchased)
3 csat_frac_viewed = df_viewed['csat_answered'].sum() / len(df_viewed)
4
5 print(f'''покрытие ксат по всей выборке: {csat_frac:.3f}''')
6 print(f'''покрытие ксат по купившим: {csat_frac_purchased:.3f}''')
7 print(f'''покрытие ксат по просмотревшим: {csat_frac_viewed:.3f}''')

```

покрытие ксат по всей выборке: 0.079
 покрытие ксат по купившим: 0.400
 покрытие ксат по просмотревшим: 0.101

Видим значительный перекося в сторону тех, кто покупает товар

Если мы хотим оценить ксат по товарам, то в таком случае можно смотреть только ксат по купившим, и никакой проблемы не будет. А вот если мы хотим смотреть, например, ксат по сайту, то в таком случае лучше смотреть ксат по всем, причем перевзвешенный с весами долей сегментов.

Ввод [43]:

```

1 df_not_viewed = df[~df['viewed']]
2 df_viewed_not_purchased = df[(df['viewed']) & (df['purchased'])]
3 df_purchased = df[df['purchased']]
4
5
6 csat_not_viewed = df_not_viewed['csat_score'].mean()
7 csat_viewed_not_purchased = df_viewed_not_purchased['csat_score'].mean()
8 csat_purchased = df_purchased['csat_score'].mean()
9
10 frac_not_viewed = len(df_not_viewed) / len(df)
11 frac_viewed_not_purchased = len(df_viewed_not_purchased) / len(df)
12 frac_purchased = len(df_purchased) / len(df)

```

Ввод [44]:

```

1 csats = [csat_not_viewed, csat_viewed_not_purchased, csat_purchased]
2 weights = [frac_not_viewed, frac_viewed_not_purchased, frac_purchased]
3
4 csat_right = np.average(csats, weights=weights)
5 print(f'''перевзвешенный ксат по просмотревшим: {csat_right:.2f}''')

```

перевзвешенный ксат по просмотревшим: 3.17

Вывод: мы встретили ошибку выжившего в данных CSAT: посчитали среднюю оценку, оценку по группам и перевзвешенную оценку с весами сегментов, которая значительно отличается от изначального среднего. Это значит, что данные по CSAT нельзя считать достоверными: у них есть явное смещение.

2.3 Эффективность расчета

Ситуация: Команда хочет измерять удовлетворённость пользователей (CSAT) с помощью развёрнутой формы-опроса (10+ вопросов) в мобильном приложении и сразу раскатить её на всех пользователей.

Вопрос в зал: Стоит ли внедрять такую длинную форму сразу для всех?

Ответ: Не стоит.

- **Риск:** длинная анкета вызовет раздражение и негатив, что может привести к оттоку и прямым денежным потерям.
- **Рекомендация:** сначала протестировать на небольшой контрольной группе (например, 1–2% пользователей), измерить:
 - уровень заполнения (отклик),
 - изменение ключевых KPI (ретеншн, ARPU),
 - чистые денежные убытки/выгоды.

Если ценность инсайтов (улучшения продукта, рост удержания/дохода) превышает потери, постепенно расширять форму на всё больше пользователей.

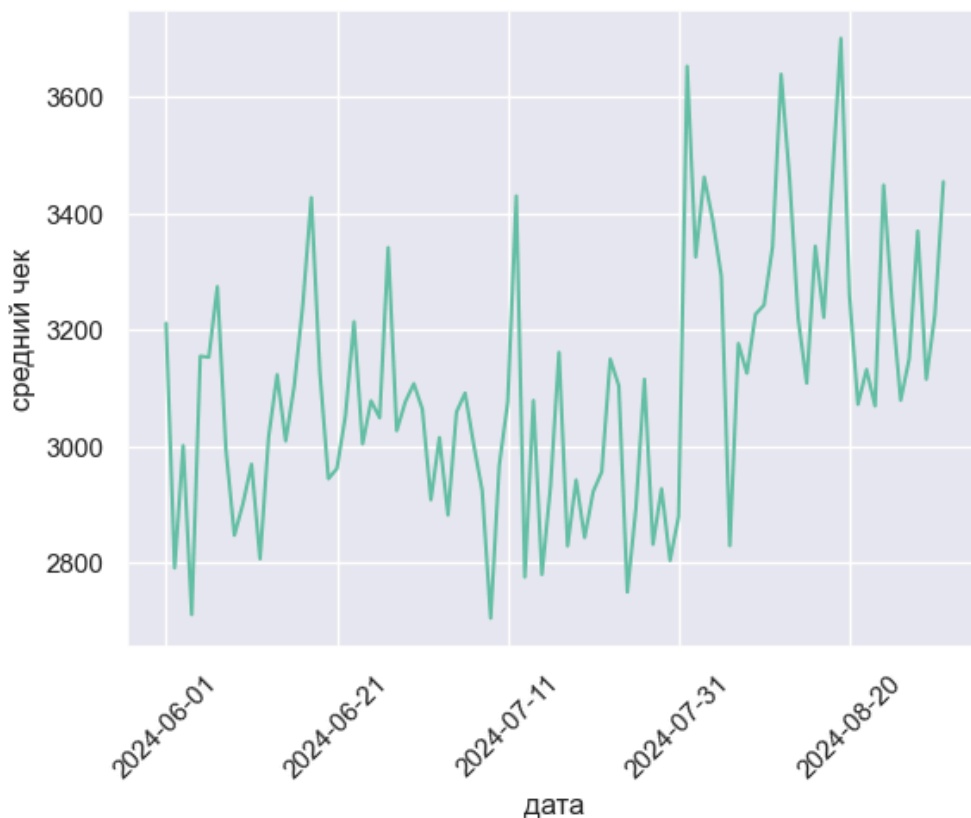
2.4 Отлаживаемость и интерпретируемость

Ситуация: К нам приходит продакт со словами: Ура! Мы провели АБ-тест в августе и заметили повышение среднего чека, посмотри на график (график ниже)

Вопрос в зал: Что делаем? Что отвечаем?

Ввод [84]:

```
1 plt.plot(figsize=(10, 5))
2 df_purchased = df[df['purchased']]
3 df_purchased.groupby('session_date')['spend'].mean().plot()
4 plt.xticks(rotation=45);
5 plt.xlabel('дата')
6 plt.ylabel('средний чек');
```



Ответ студентов: скорее всего, ответят про дов. интервалы: не факт, что разница стат. значима.

Давайте построим дов. интервалы

Ввод [89]:

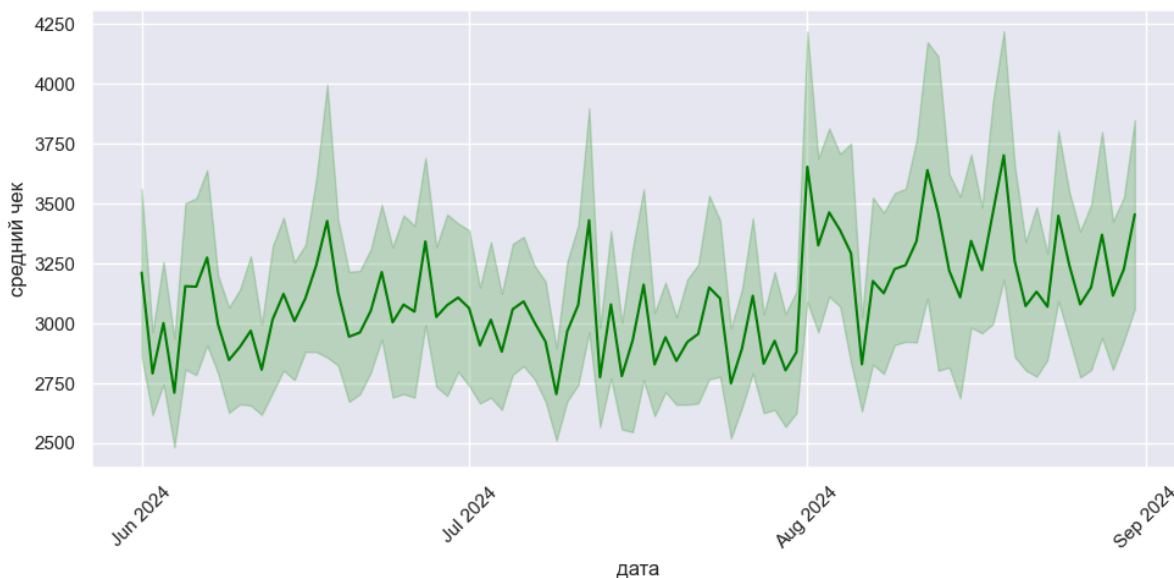
```

1  # Группировка и расчёты
2  z = sps.norm.ppf(0.975)
3  df_purchased["session_date"] = pd.to_datetime(df_purchased["session_date"])
4  df_grouped = df_purchased.groupby("session_date")["spend"].mean()
5  df_grouped["cnt"] = df_purchased.groupby("session_date")["spend"].count()
6  df_grouped["std"] = df_purchased.groupby("session_date")["spend"].std() / np.s
7  df_grouped["lb"] = df_grouped["spend"] - z * df_grouped["std"]
8  df_grouped["rb"] = df_grouped["spend"] + z * df_grouped["std"]
9  df_grouped = df_grouped.reset_index()
10
11 # Построение графика
12 fig, ax = plt.subplots(figsize=(10, 5))
13 ax.plot(df_grouped["session_date"], df_grouped["spend"], color="green")
14 ax.fill_between(df_grouped["session_date"], df_grouped["lb"], df_grouped["rb"]
15
16 # Настройка оси X: тики по месяцам
17 ax.xaxis.set_major_locator(mdates.MonthLocator())
18 ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
19 plt.xticks(rotation=45)
20
21 ax.set_xlabel("дата")
22 ax.set_ylabel("средний чек")
23 plt.tight_layout()
24 plt.show()

```

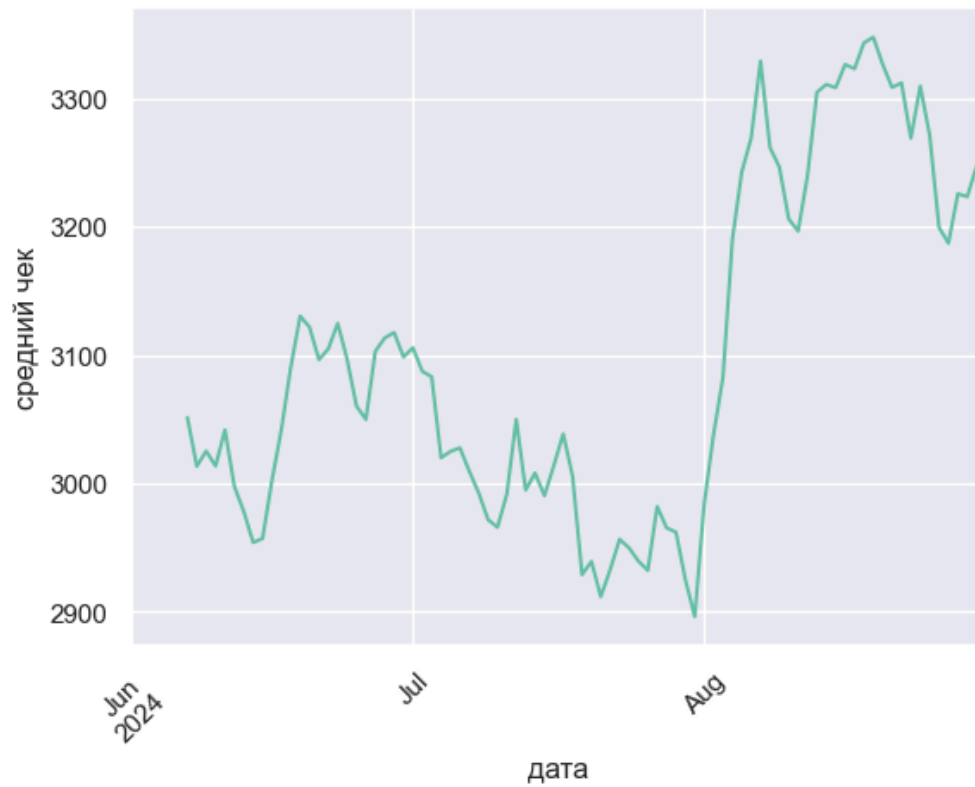
/var/folders/ym/2l1gn_s5ss0_f4qd4t25z500000gq/T/ipykernel_7849/464916299.py:3: Set
tingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)
df_purchased["session_date"] = pd.to_datetime(df_purchased["session_date"])



Попробуем посчитать скользящее среднее за неделю и убедимся, что средний чек действительно вырос.

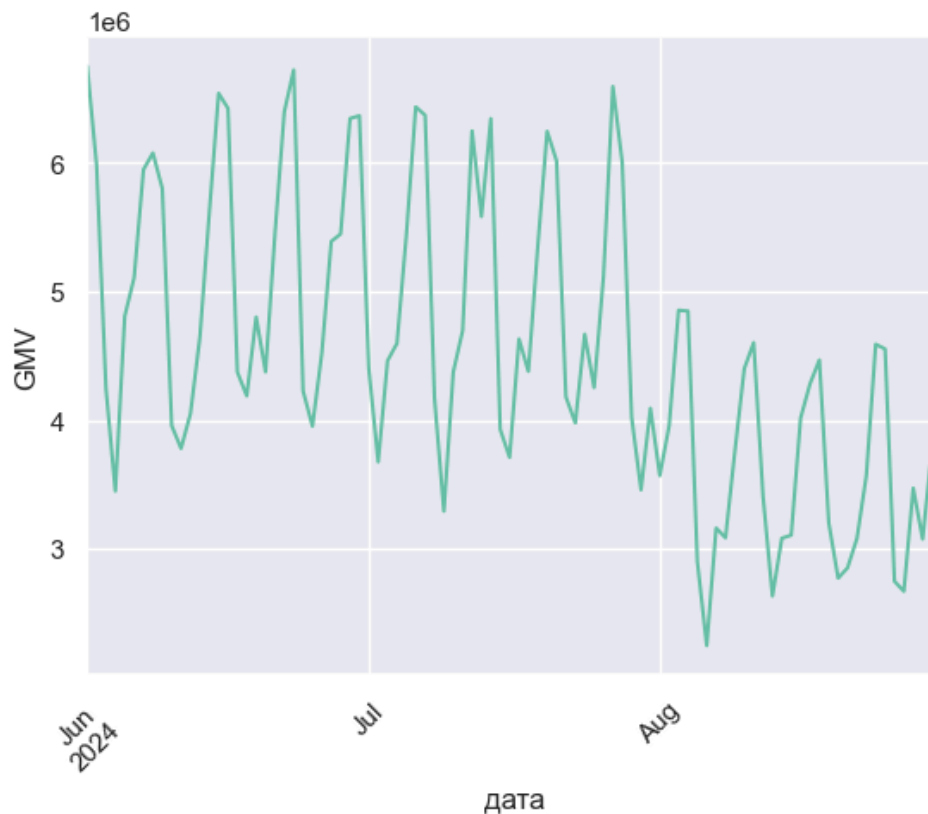
```
Ввод [93]: 1 df_grouped['rolling_mean'] = (df_grouped['spend'] * df_grouped['cnt']).rolli
2 df_grouped.set_index('session_date')['rolling_mean'].plot()
3 plt.xticks(rotation=45);
4 plt.xlabel('дата')
5 plt.ylabel('средний чек');
```



Вопрос в зал: Кажется, действительно тест успешен. Может ли что-то пойти не так?

Ответ: стоит посмотреть на GMV по дням, то есть, на суммарную подневную прибыль.

```
Ввод [95]: 1 df_grouped['GMV'] = (df_grouped['spend'] * df_grouped['cnt'])
2 df_grouped.set_index('session_date')['GMV'].plot()
3 plt.xticks(rotation=45);
4 plt.xlabel('дата')
5 plt.ylabel('GMV');
```



Итак, подневная прибыль упала. Давайте разбираться, что пошло не так:

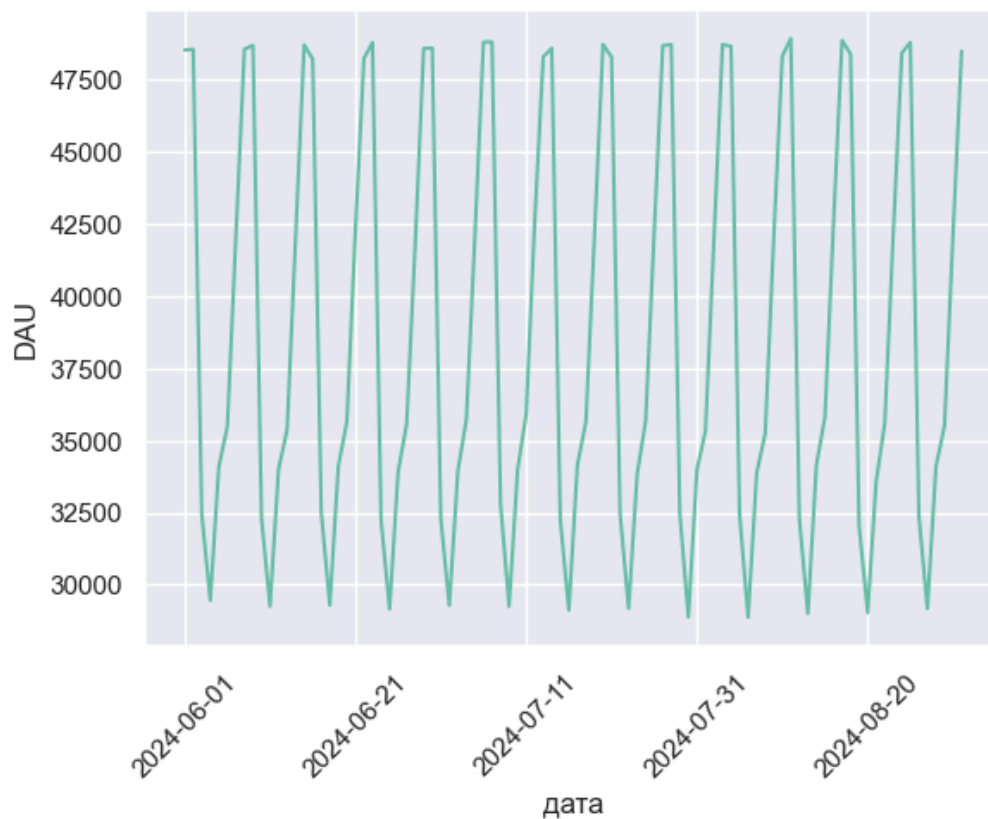
$GMV = AOV \times \text{кол-во покупок}$.

Мы уже видели, что средний чек даже вырос значит, упало кол-во покупок. Разбираемся дальше:

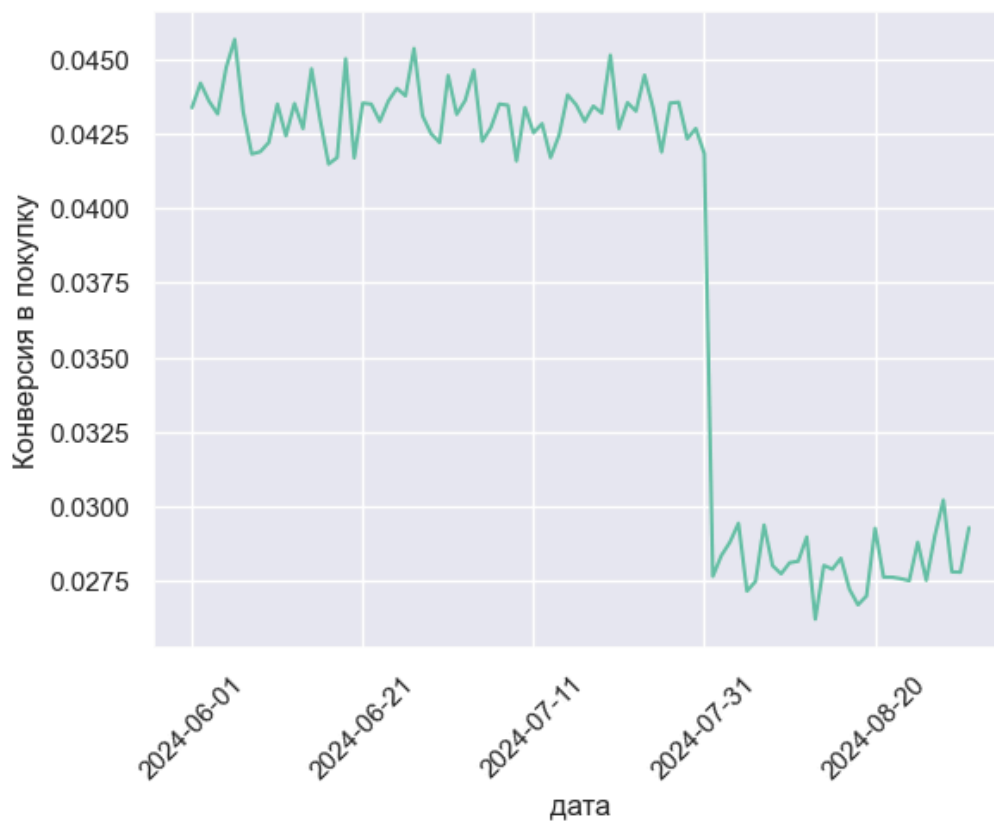
$\text{кол-во покупок} = DAU \times \text{конверсия в покупку}$.

Давайте посмотрим на посещаемость и конверсию.

```
Ввод [97]: 1 df.groupby('session_date')['session_id'].count().plot()  
2 plt.xticks(rotation=45);  
3 plt.xlabel('дата')  
4 plt.ylabel('DAU');
```



```
Ввод [98]: 1 df.groupby('session_date')['purchased'].mean().plot()  
2 plt.xticks(rotation=45);  
3 plt.xlabel('дата')  
4 plt.ylabel('Конверсия в покупку');
```

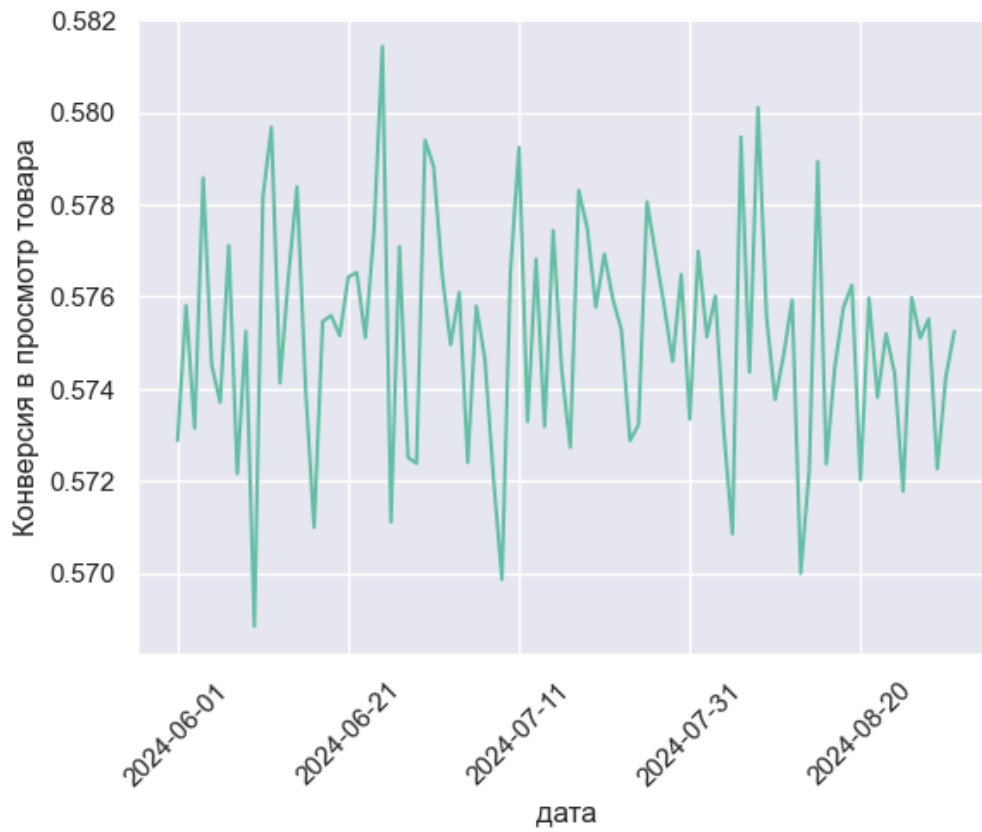


Так вот в чем дело: резко упала конверсия в покупку!

Вопрос в зал: Что же произошло?

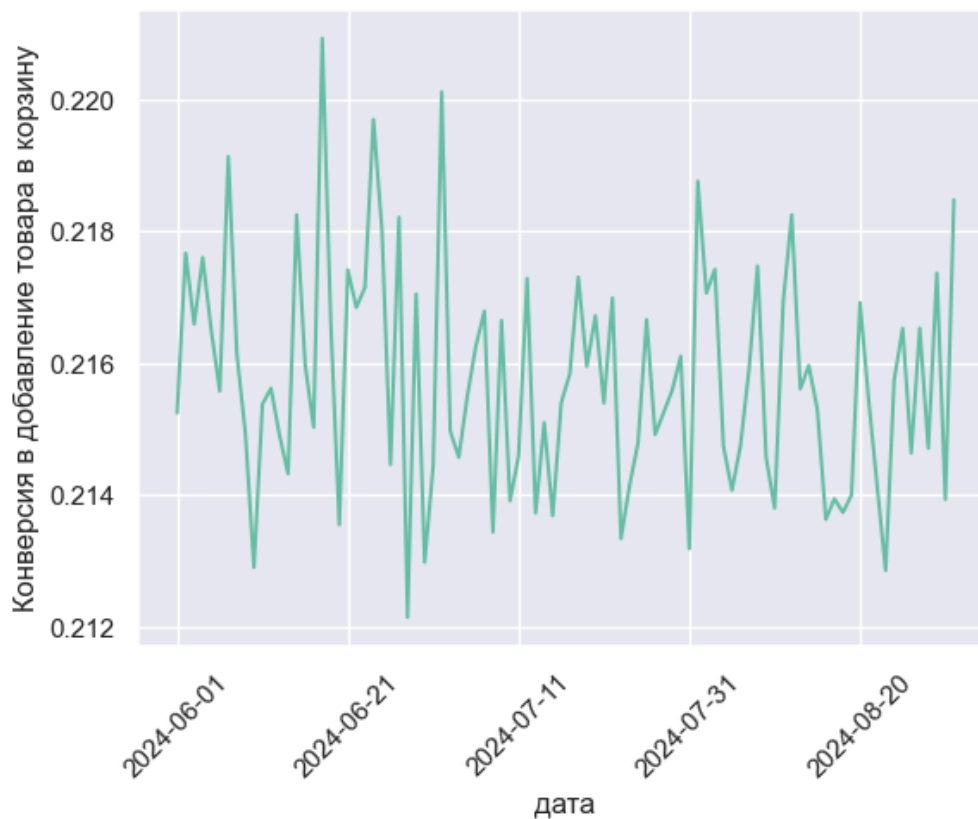
Ответ: давайте посмотрим другие конверсии: в просмотр и в добавление в корзину.

```
Ввод [99]: 1 df.groupby('session_date')['viewed'].mean().plot()  
2 plt.xticks(rotation=45);  
3 plt.xlabel('дата')  
4 plt.ylabel('Конверсия в просмотр товара');
```



Ввод [100]:

```
1 df.groupby('session_date')['added'].mean().plot()  
2 plt.xticks(rotation=45);  
3 plt.xlabel('дата')  
4 plt.ylabel('Конверсия в добавление товара в корзину');
```



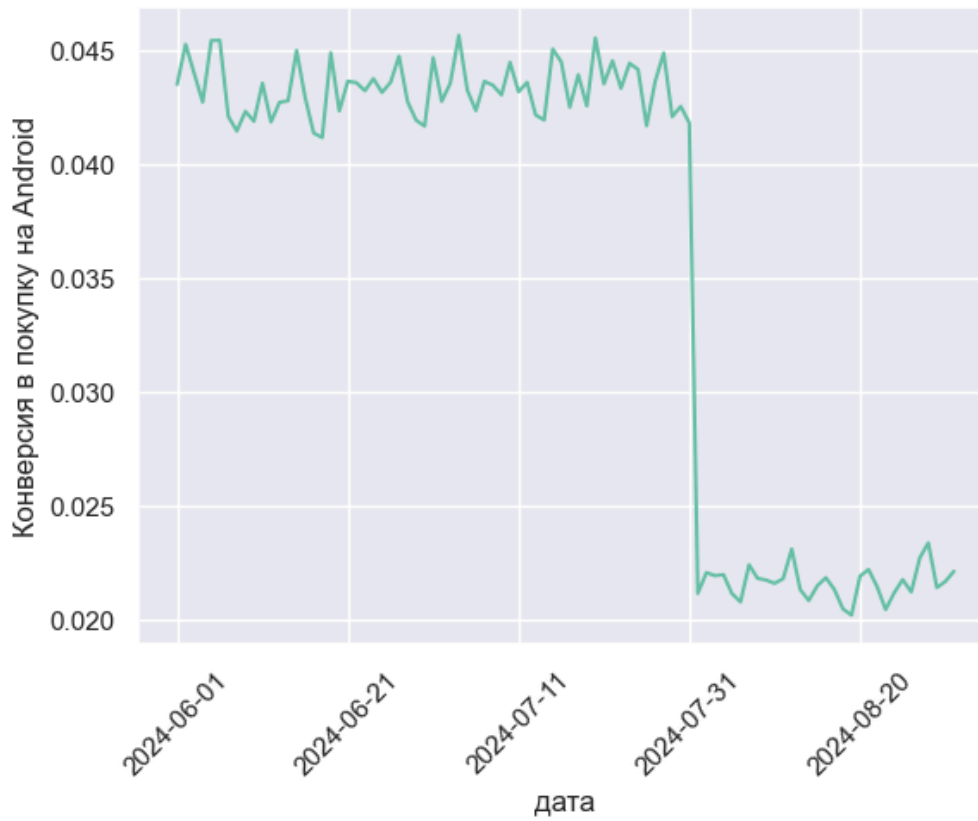
Эти конверсии не изменились. Скорее всего, у нас произошел сбой в покупках. Давайте разбираться дальше, что произошло: странно, что при этом вырос средний чек.

Вопрос в зал: Идеи?

Ответ: давайте посмотрим конверсию в покупку в разрезах ОС.

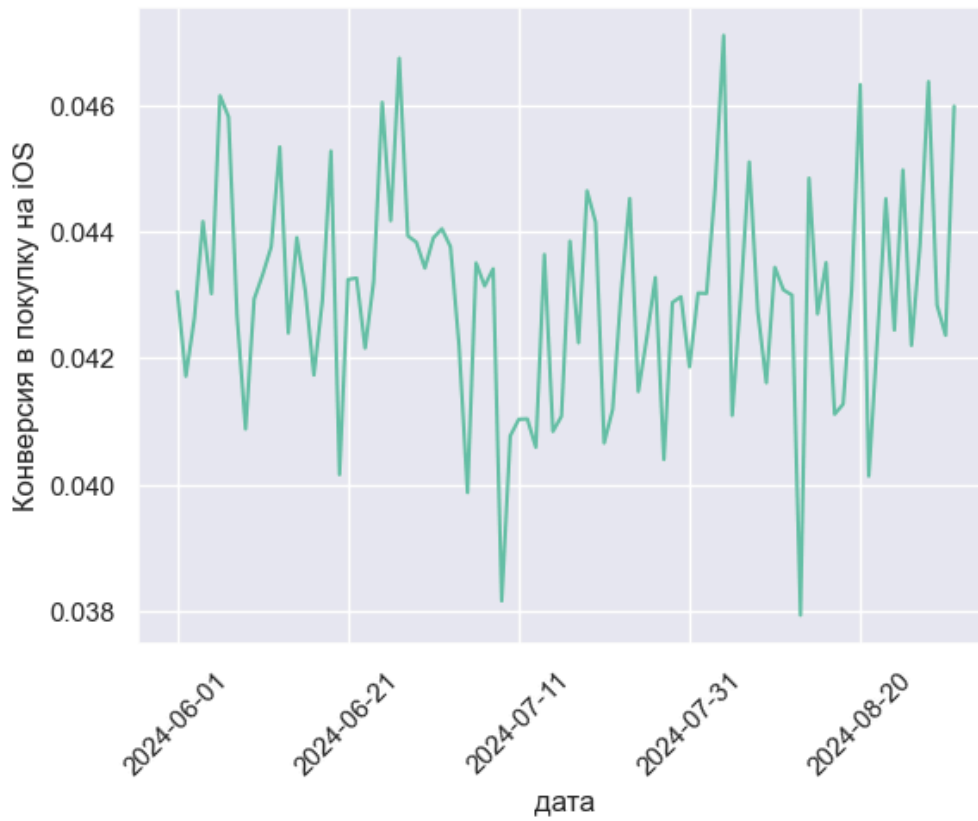
Ввод [101]:

```
1 df_android = df[df['os'] == 'Android']  
2  
3 df_android.groupby('session_date')['purchased'].mean().plot()  
4 plt.xticks(rotation=45);  
5 plt.xlabel('дата')  
6 plt.ylabel('Конверсия в покупку на Android');
```



Ввод [102]:

```
1 df_ios = df[df['os'] == 'iOS']  
2  
3 df_ios.groupby('session_date')['purchased'].mean().plot()  
4 plt.xticks(rotation=45);  
5 plt.xlabel('дата')  
6 plt.ylabel('Конверсия в покупку на iOS');
```



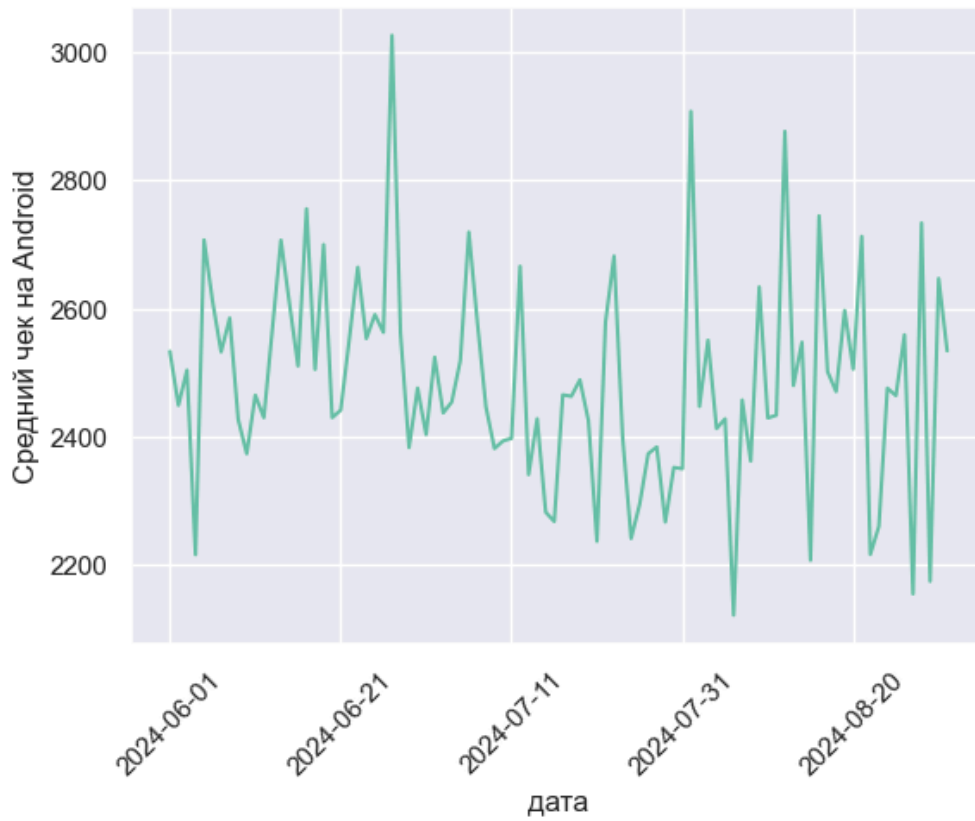
Конверсия в покупку на андроиде упала примерно в 2 раза. На практике произошло что-то из двух:

- мы что-то сломали, поэтому покупки на Android перестали стабильно работать
- с покупками все нормально, а вот наши данные на Android льются в систему не полностью

Вопрос в зал: А почему же средний чек вырос?

Ответ: давайте посмотрим средний чек в разрезах ОС.

```
Ввод [103]: 1 df_purchased_android = df_android[df_android['purchased']]  
2  
3 df_purchased_android.groupby('session_date')['spend'].mean().plot()  
4 plt.xticks(rotation=45);  
5 plt.xlabel('дата')  
6 plt.ylabel('Средний чек на Android');
```

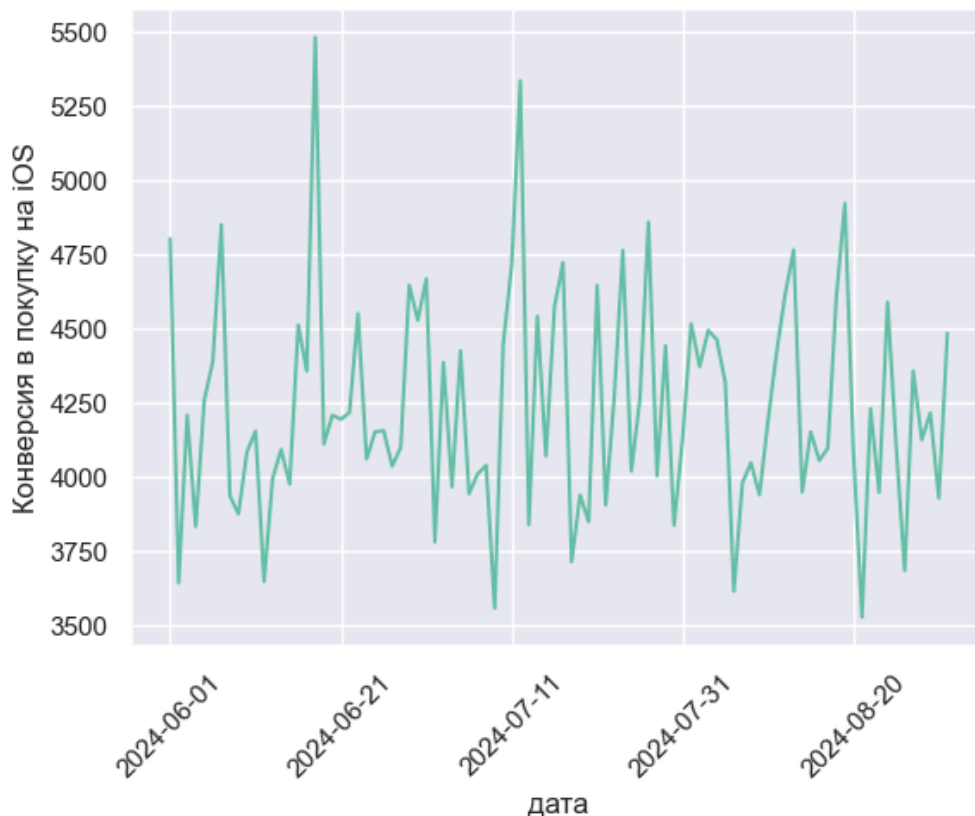


Ввод [104]:

```

1 df_purchased_ios = df_ios[df_ios['purchased']]
2
3 df_purchased_ios.groupby('session_date')['spend'].mean().plot()
4 plt.xticks(rotation=45);
5 plt.xlabel('дата')
6 plt.ylabel('Конверсия в покупку на iOS');

```



Средний чек по ОС не поменялся, но в на iOS он в среднем всегда выше, а покупки на андроид сломались, поэтому и общий средний чек вырос.

Вывод по задаче: мы разобрали кейс, где декомпозировали различные метрики и разобрались, почему же произошло изменение метрик после теста. Несмотря на рост среднего чека, прибыль упала, так что в данном случае рекомендуется устранить баг (либо в системе, либо в проливании данных) на Android, а после — повторить тест.

Вывод по свойствам:

- в данном случае средний чек не показал себя как интерпретируемая и плохо отлаживаемая метрика:
 - интерпретируемость отсутствует, так как к нам пришел продакт с мыслью "средний чек вырос, значит, круто, катим изменения", а на самом деле оказалось, что GMV упал, так что изменения ни в коем случае нельзя катить
 - отлаживаемость низкая, так как в данном случае мы не можем декомпозировать средний чек. Да, нам помогла сегментация по ОС, и мы увидели, что средний чек по ОС не изменился, однако при его изменении мы бы не смогли понять, что повлияло: инфляция, изменение типа клиента или что-то еще
- напротив, GMV показало себя как интерпретируемая и хорошо отлаживаемая метрика:
 - интерпретируемость есть, так как все понимают, что рост GMV — это хорошо
 - отлаживаемость высокая, так как мы смогли декомпозировать GMV и выяснить причину падения
- число заказов и конверсия в покупку показали здесь — это хорошо отлаживаемые метрики, но не всегда интерпретируемые:
 - интерпретируемость может быть низкой. Пример: мы выходим на рынок дорогих клиентов/продуктов, и конверсия падает. GMV при этом может значительно вырасти из-за большого

- роста среднего чека
- отлаживаемость высокая, так как мы смогли декомпозировать эти метрики и выяснить причину падения

2.5. Общий вывод по задаче

Идеальной метрики под все задачи не существует. Какие-то метрики показывают себя хорошо в одном случае и плохо в другом. Важно **выбирать метрику непосредственно под задачу**. Например, если у нас продукт только запустился, то для нас были бы важнее не высокие конверсии в покупку, а больший охват аудитории.

3. Выбор метрики под конкретную задачу

3.1 🍷 Практика. Задача 1

Задача

Ты работаешь в стартапе мобильной игры, который только что запустился. Нужно замерить эффективность рекламной кампании. При этом есть две разные рекламные кампании, и ты хочешь провести A/B тест, случайно разделив пользователей на две группы, чтобы определить, какая кампания эффективнее. Предложи, какую метрику использовать для оценки эффективности и как сравнить кампании.

Решение

Для оценки эффективности рекламных кампаний в стартапе мобильной игры на начальном этапе я бы рекомендовал сосредоточиться на метрике **конверсии из показа рекламы в заход в приложение**.

Пояснение:

- Мы смотрим не просто клики, а сколько из всех пользователей, которым была показана реклама, реально запустили игру. Это позволяет оценить реальную эффективность рекламного охвата и качество трафика.
- Так как планируется провести A/B тестирование с рандомным разделением пользователей на две группы, использование этой метрики обеспечит объективное и справедливое сравнение.

Для сравнения кампаний:

- Пользователи случайно делятся на две группы, каждая из которых видит свою рекламную кампанию.
- По каждой группе измеряется **конверсия из показа в заход**.
- Кампания с более высокой конверсией считается более эффективной.
- Если есть возможность, дополнительно стоит анализировать метрики удержания (Retention) и монетизации (ARPU, LTV), чтобы оценить не только количество пользователей, но и их качество.

3.1 🍷 Практика. Задача 2

Задача

Ты аналитик в Wildberries. Ваша команда хочет протестировать новую версию рекомендательной системы, которая, как ожидается, будет лучше подбирать товары для конкретного пользователя. Нужно предложить метрику для оценки и сравнения качества рекомендаций.

Решение

Для оценки качества новой рекомендательной системы я бы предложил использовать **конверсию в покупку** как основную метрику.

Пояснение:

- Целевое действие пользователя — это покупка товара, поэтому именно конверсия в покупку наиболее точно отражает эффективность рекомендаций.
- Если новая система предлагает релевантные и интересные товары, пользователи с большей вероятностью совершат покупку, что и покажет повышение конверсии.
- Дополнительно следует отслеживать **средний чек** (average order value), чтобы убедиться, что увеличение конверсии не происходит за счет снижения стоимости покупок. В идеале средний чек должен оставаться на прежнем уровне или расти, что будет свидетельствовать о повышении качества рекомендаций с точки зрения бизнеса.

4. Итоги семинара

С чем выходим с занятия:

- повторили свойства метрик
- отметили, что метрики могут показывать себя по-разному в зависимости от ситуации
- посмотрели на практике, что означают некоторые свойства
- поняли, что идеальных метрик не бывает
- попрактиковались выбирать метрику под задачу